



Intro Machine Learning

CHAPTERS

- Machine Learning Applications
- Learning Paradigms in ML
- Metrics

~3 contact hours

WHY THIS MODULE MATTERS

Provides a general introduction to machine learning and main evaluation metrics

Machine Learning



“An important feature of a **learning machine** is that its teacher will often be very largely ignorant of quite what is going on inside [...]. The learning process may be regarded as a search for a form of behaviour which will satisfy the teacher (or some other criterion).”

A. Turing

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T , as measured by P , improves with experience E .”

T. Mitchell

Machine Learning



Machine Learning is currently one of the **most dynamic and transformative areas** within Artificial Intelligence and Computer Science.

Leading global ICT companies—including **Google, Meta (Facebook), IBM, Baidu, Amazon, Microsoft, and Disney**—are heavily investing in Machine Learning technologies.

Why ?

Machine Learning



Because these companies generate **massive volumes of data**—and Machine Learning provides the tools to **extract value, automate decisions, and create intelligent services**.

Machine Learning: Spam Detection

Supervised learning is used to train models on labeled datasets of emails (spam vs. not spam).

Algorithms learn to recognize patterns in:

- Email content (keywords, phrases)
- Sender behavior
- Metadata (e.g., frequency, links, attachments)

Real-world impact:

Spam filters in Gmail, Outlook, and other platforms now block **over 99% of unwanted emails**, improving productivity and security.



Figure source: <http://www.resilientsystems.co.uk/>

Machine Learning: Forecasting Stock Prices

Machine Learning models can be trained to **predict short-term movements** in stock prices, such as Apple's, using historical data and real-time market signals.

Real-world use:

Used by trading platforms, hedge funds, and financial analysts to support decision-making and automate trading strategies.

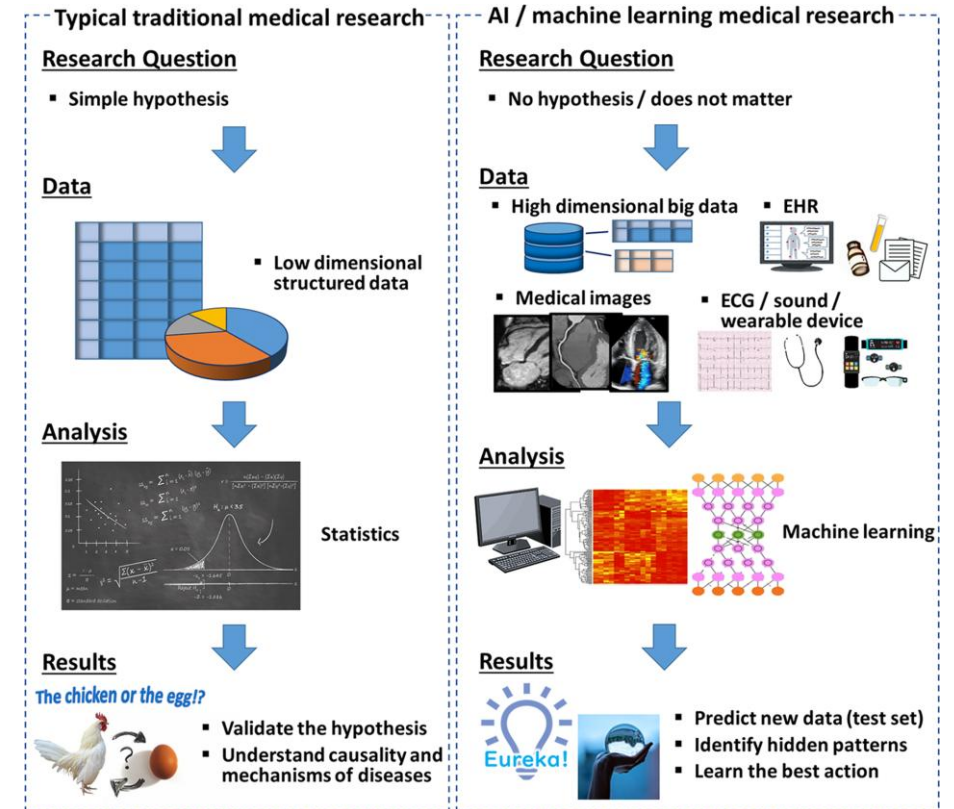


Machine Learning: Medical Diagnosis

- ML models are trained on large datasets containing **patient records**, **lab results**, **medical images**, and **clinical notes**.
- Algorithms learn to identify patterns and correlations that may not be obvious to human clinicians.

Real world applications

- Early detection of diseases like cancer, diabetes, and heart conditions.
- Support for doctors in making faster, more accurate decisions.
- Reduction in diagnostic errors and improved patient outcomes.



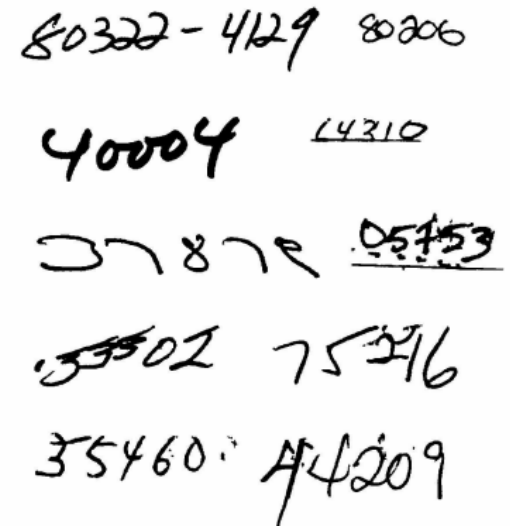
Machine Learning: Digit Recognition in Zipcode Forms

One of the classic applications of Machine Learning is **handwritten digit recognition**, used in postal systems to automatically read zip codes.

One of the classic applications of Machine Learning is **handwritten digit recognition**, used in postal systems to automatically read zip codes.

Real-world impact:

- Speeds up mail sorting and delivery.
- Reduces manual errors and improves efficiency in logistics.



Handwritten zip codes from a postal form, showing examples of digits and their corresponding zip codes. The text is written in black ink on a white background. The zip codes are: 80322-4129 80206, 40004 14310, 37879 05453, 3302 75216, and 35460 44209.

Figure source: [LeCun et al., 1989]

Machine Learning: Robotic Grasping

Machine Learning enables robots to learn how to **grasp objects**—like a mug—by interpreting sensory data and adapting to different shapes, weights, and positions.

How it works:

- Robots use **camera vision** and **depth sensors** to perceive the environment.
- ML models learn from trial and error how to position the gripper and apply the right force.
- Training involves **simulations** and **real-world feedback loops**.

Applications:

- Industrial automation (e.g., pick-and-place tasks)
- Service robots (e.g., assistive technologies)
- Warehousing and logistics



Figure source: <http://www.informatik.uni-bremen.de/>

Machine Learning: Drug and Molecule Design

Machine Learning is transforming pharmaceutical research by enabling the **design of new drugs and molecules** with specific biological or chemical properties.

How it works:

- ML models analyze vast datasets of molecular structures and known drug interactions.
- Algorithms learn to **predict molecular behavior**, such as binding affinity, toxicity, and solubility.
- Generative models can create entirely new molecular structures optimized for desired properties.

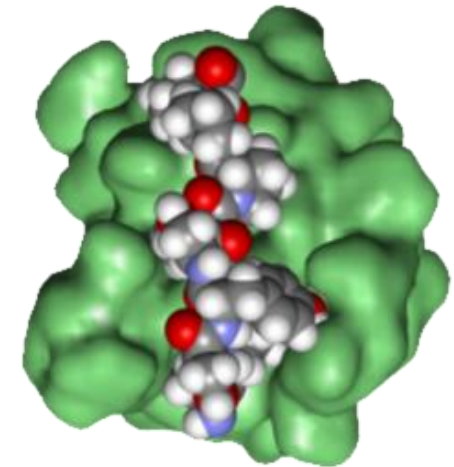


Figure source: <http://pande.stanford.edu>

Machine Learning: Machine Translation

How it works:

- ML models are trained on **parallel corpora**—large datasets of aligned text in multiple languages.
- Algorithms learn to map linguistic structures and meanings from one language to another.

Real-world impact:

- Used in tools like **Google Translate**, **DeepL**, and **Microsoft Translator**.
- Supports global communication, education, diplomacy, and commerce.



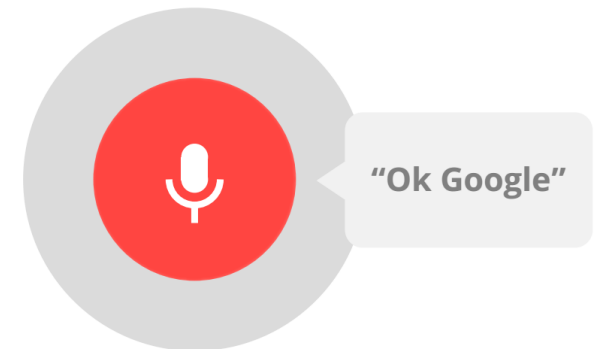
Machine Learning: Speech-to-Text

How it works:

- Audio input is processed to extract features like pitch, tone, and phonemes.
- ML models (especially deep learning) map these features to words and sentences.

Real-world applications:

- Virtual assistants (e.g., Siri, Alexa, Google Assistant)
- Automated meeting transcription (e.g., Zoom)



Machine Learning: Movie Recommendation

How it works:

- ML models analyze **user behavior**, such as viewing history, ratings, and search patterns.
- They use this data to predict what content a user is likely to enjoy.

Real-world impact:

- Increases user engagement and satisfaction.
- Drives personalized experiences and content discovery.
- Helps platforms optimize content delivery and production strategies.



Machine Learning: Image Captioning

How it works:

- **Convolutional Neural Networks (CNNs)** extract visual features from the image.
- **Recurrent Neural Networks (RNNs)** or **Transformers** generate a textual description based on those features.
- The model learns from large datasets of images paired with human-written captions (e.g., MS COCO).

Applications:

- Accessibility tools for visually impaired users
- Automated content tagging and indexing
- Enhancing search engines and digital assistants

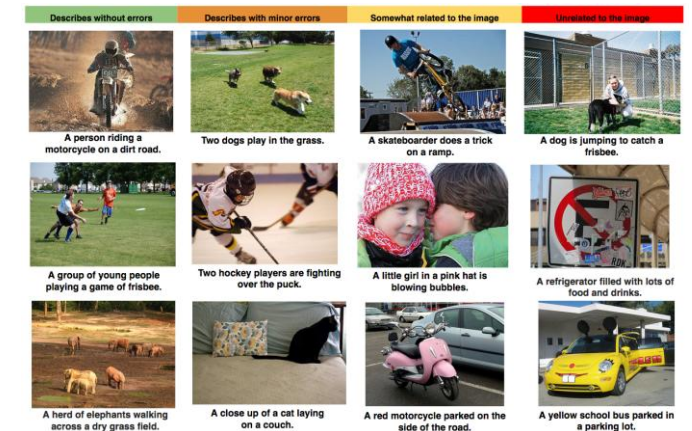


Figure source: Google Research

Machine Learning

AlphaGo Defeats the World's Top Go Player

In **March 2016**, Google DeepMind's **AlphaGo** made history by defeating **Lee Sedol**, one of the world's best Go players, in a five-game match.

- Marked a major leap in AI's ability to handle intuition, strategy, and creativity.
- Inspired further research in general-purpose learning systems.
- Sparked global interest in AI ethics, transparency, and human-AI collaboration.



Machine Learning: NLU and Reasoning

Natural Language Understanding (NLU) involves parsing and interpreting text to extract **meaning**, **intent**, and **context**.

Applications:

- Legal and medical document analysis
- Automated customer support
- Scientific research summarization
- Intelligent tutoring systems

GPT-4 Generating Template

Prompt: Andy harvests all the tomatoes from 18 plants that have 7 tomatoes each. If he dries half the tomatoes and turns a third of the remainder into marinara sauce, how many tomatoes are left?
Can you generate a template solution for the problem without numbers?

Using symbols, the template solution could look something like this:

- Let P be the number of plants and T be the number of tomatoes per plant. Then the total number of tomatoes harvested is $P \times T$.
- Let D be the number of tomatoes that are dried. Then $D = (P \times T) / 2$.
- Let M be the number of tomatoes that are turned into marinara sauce. Then $M = ((P \times T) - D) / 3$.
- Let L be the number of tomatoes that are left. Then $L = (P \times T) - D - M$.
- ...

[Figure from Bubek et al., 2023]

Main Machine Learning Paradigms

Supervised Learning

- **Goal:** Learn a mapping from inputs to known outputs.
- **Data:** Labeled examples (input-output pairs).
- **Applications:** Spam detection, image classification, medical diagnosis, stock price prediction.

Unsupervised Learning

- **Goal:** Discover hidden patterns or structures in data.
- **Data:** Unlabeled data (no predefined outputs).
- **Applications:** Customer segmentation, anomaly detection, dimensionality reduction.

Reinforcement Learning

- **Goal:** Learn to make decisions by interacting with an environment.
- **Data:** Feedback in the form of rewards or penalties.
- **Applications:** Robotics, game playing (e.g., AlphaGo), autonomous vehicles.

Supervised Learning

Find a mapping between input/output variables, directly **from data observations**

- **Observed variables** $x_i \in X$
- **Prediction targets** $y_i \in Y$
- **Data set**, a collection of (x_i, y_i) pairs
- Build a **function** $f : X \rightarrow Y$
- Function f must be able to **predict** $\hat{y}$ given some $\hat{x}$

Supervised Learning



Different methods and problems according to:

- the kind of **input** space X
- the kind of **output** space Y
- the choices made for the **definition** of function f
- the choices made for the **computation** of function f

Supervised Learning



Classic input spaces

- X = numerical and/or attributes
- X = logic representations
- X = some structured space (tree, graph, etc.)

Supervised Learning



Classic output spaces

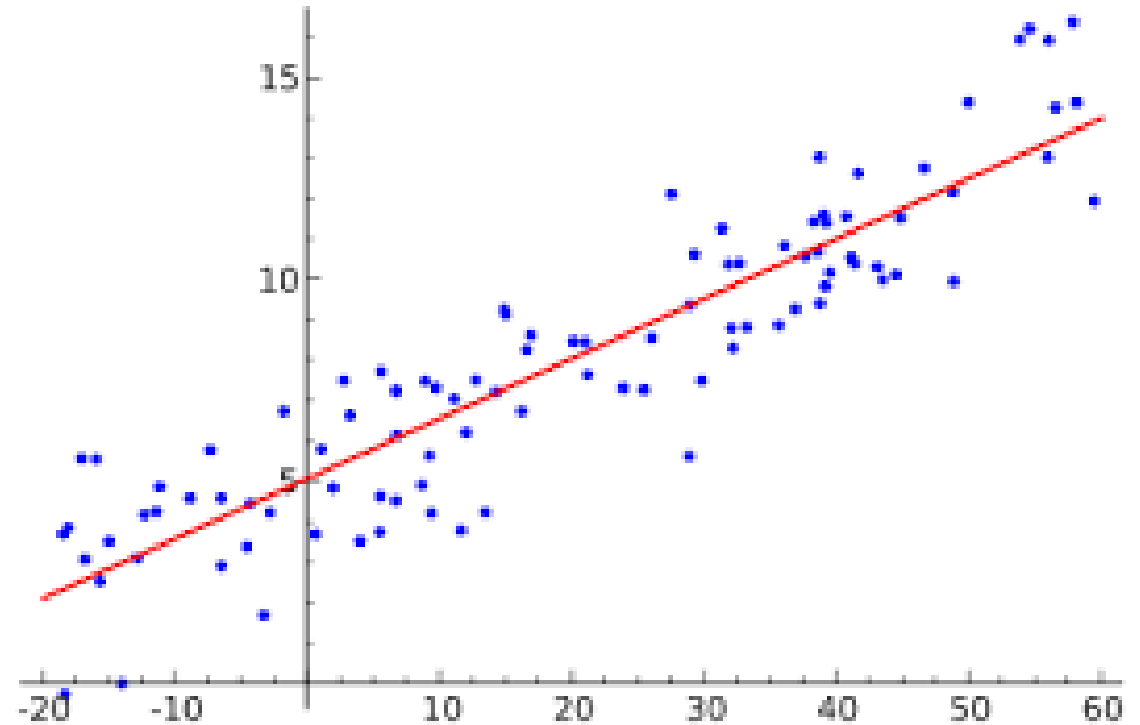
- $Y = \{0, 1\}$ for binary classification
- $Y = \{1, \dots, M\}$ for multi-class classification
- $Y = \mathbb{R}$ for regression
- $Y =$ some structured space for structured output

Supervised Learning

A classic example: linear regression

- $\mathbf{X} = \mathbb{R}^d$
- $Y = \mathbb{R}$
- $y = f(x) = \sum_{i=1}^d \alpha_i x_i$
- $\alpha = \arg \min \sum_{i=1}^N (\hat{y}_i - y_i)^2$

[Image from Wikipedia]



Unsupervised Learning

- No labeled data or predefined outputs.
- The task may be **unknown**—the goal is to **discover structure** in the data.

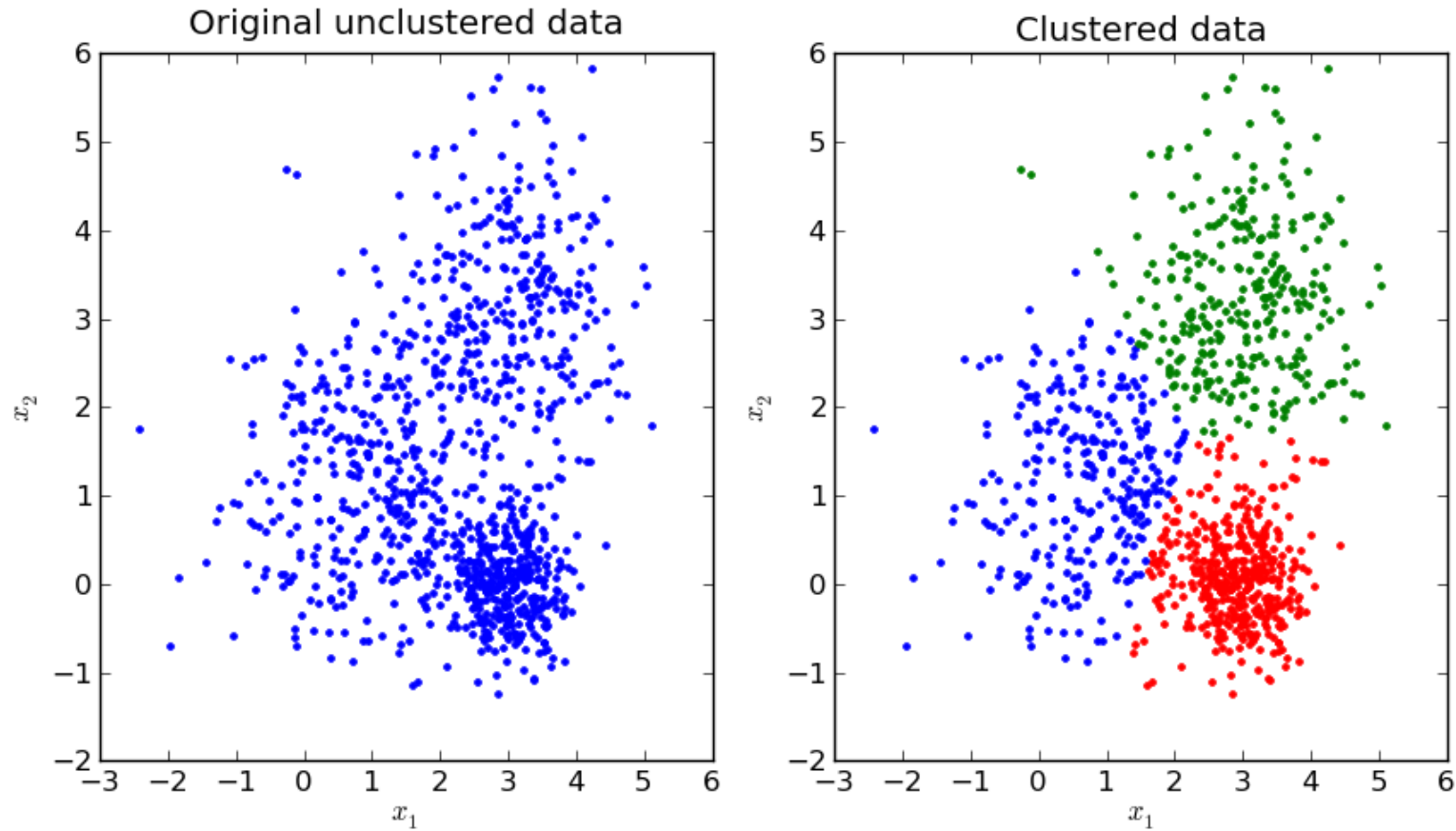
Why is it useful?

- **Cluster data** into meaningful groups (e.g., customer segmentation).
- **Find frequent patterns** or associations (e.g., market basket analysis).
- **Identify key features** that explain variability.
- **Reduce dimensionality** for visualization or preprocessing (e.g., PCA).

Common Techniques:

- K-Means Clustering
- Hierarchical Clustering
- Principal Component Analysis (PCA)
- Autoencoders

Unsupervised Learning



[Image from pypr.sourceforge.net]

Measuring Performance



Given a **problem** and a machine learning **system** that solves it, it is crucial to **measure its performance**

Idea: **count the errors** of the system...
...Not so straightforward as one could expect!

Measuring Performance



Example

We have to build a **diagnosis system** for a very **rare pathology**, based on machine learning

- Our system correctly classifies 99.9% of the cases
- Can we be happy with this classifier?

Confusion Matrix

		True Class	
		0	1
Pred Class	0	TN True Negative	FN False Negative
	1	FP False Positive	TP True Positive

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$F_1 = \frac{2PR}{P+R}$$

Confusion Matrix

		True Class		
		A	B	C
Pred Class	A	TP (A)	FP (A) FN (B)	FP (A) FN (C)
	B	FP (B) FN (A)	TP (B)	FP (B) FN (C)
	C	FP (C) FN (A)	FP (C) FN (B)	TP (C)

$$\text{Accuracy} = \frac{\sum_i TP_i}{\sum_i TP_i + FN_i} = \frac{\sum_i TP_i}{\sum_i TP_i + FP_i}$$

$$\text{Precision}^i = \frac{TP_i}{TP_i + FP_i}$$

$$\text{Recall}^i = \frac{TP_i}{TP_i + FN_i}$$

$$F_1^i = \frac{2P^i R^i}{P^i + R^i}$$

Esercizio

What is best?

A)

Accuracy 91,7%

		True Class	
		N	Y
Pred Class	N	5	0
	Y	1	6

B)

Accuracy 71,4%

		True Class	
		N	Y
Pred Class	N	400	350
	Y	50	600

C)

Accuracy 71,4%

		True Class	
		N	Y
Pred Class	N	400	50
	Y	350	600

D)

Accuracy 85,7%

		True Class	
		N	Y
Pred Class	N	600	100
	Y	0	0

ROC and RP Curves



One can compute:

- Recall-Precision (RP) Curve
- Receiver Operating Characteristics (ROC) Curve

(RP) and (ROC) curves show how a classifier's performance changes as you vary the **decision threshold**.

ROC and RP Curves



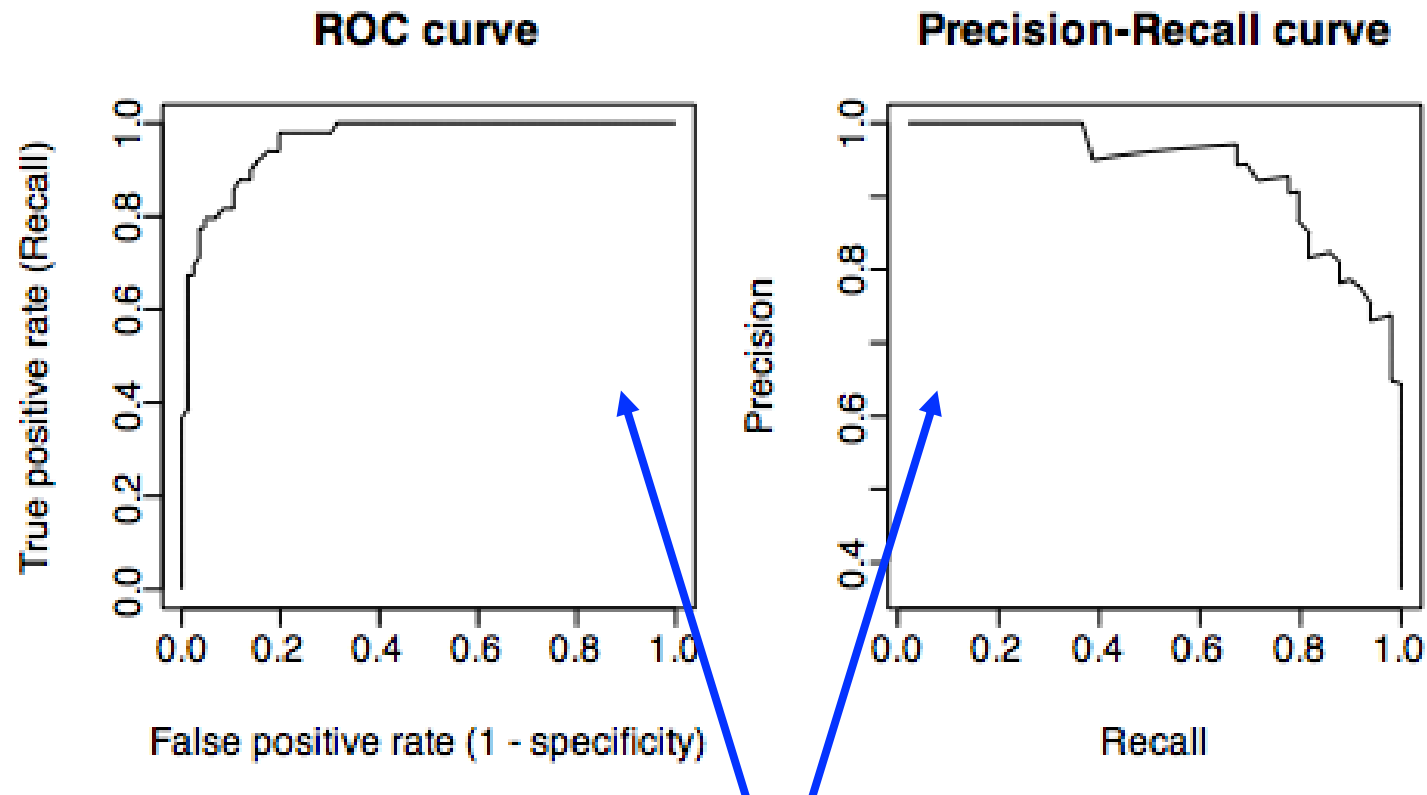
For **RP curves**

- Precision on y-axis
- Recall on x-axis

For **ROC curves**

- False Positive Rate or $FPR = FP/(FP+TN)$ on x-axis
- True Positive Rate or TPR (Recall) on y-axis

ROC and RP Curves



COMPUTE THE AREA UNDER THE CURVE (AUC)

Regression Metrics

For regression problems

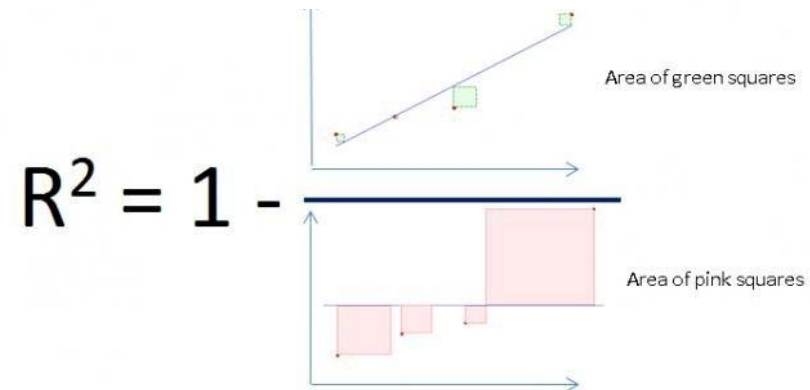
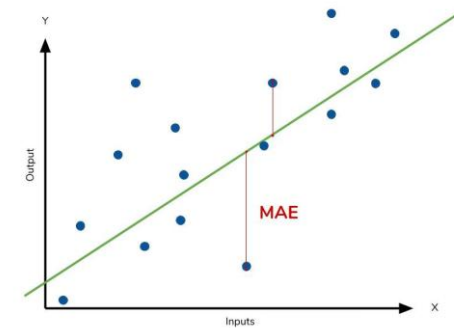
- Mean squared error (MSE)
- Root mean squared error (RMSE)

$$MSE = \sum_{i=1}^N \frac{(\hat{y}_i - y_i)^2}{N}$$

$$RMSE = \sqrt{MSE}$$

$$MAD = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

- Mean absolute deviation (MAD)
- Mean absolute error (MAE)

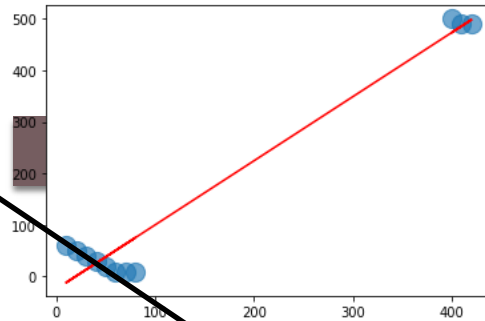


- R^2 Explained Variance

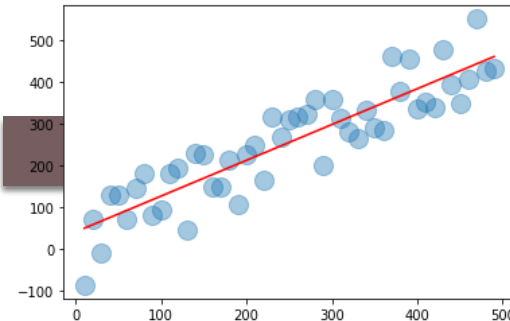
Esercizio

What is best?

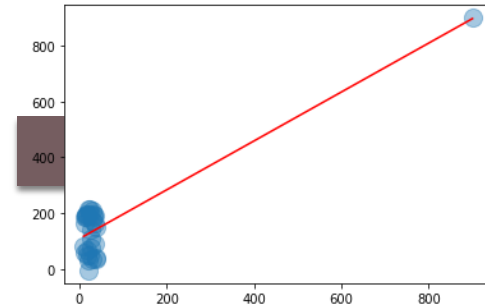
MAE = 33.5
R2 = 0.96



MAE = 52.1
R2 = 0.80



MAE = 61.96
R2 = 0.82



Model Selection



The performance of any machine learning system **strongly depends** on a set of **hyper-parameters**, whose values have to be chosen during the training phase

The **procedure** through which we choose the best hyper-parameters for a machine learning system is called **model selection**

Model Selection



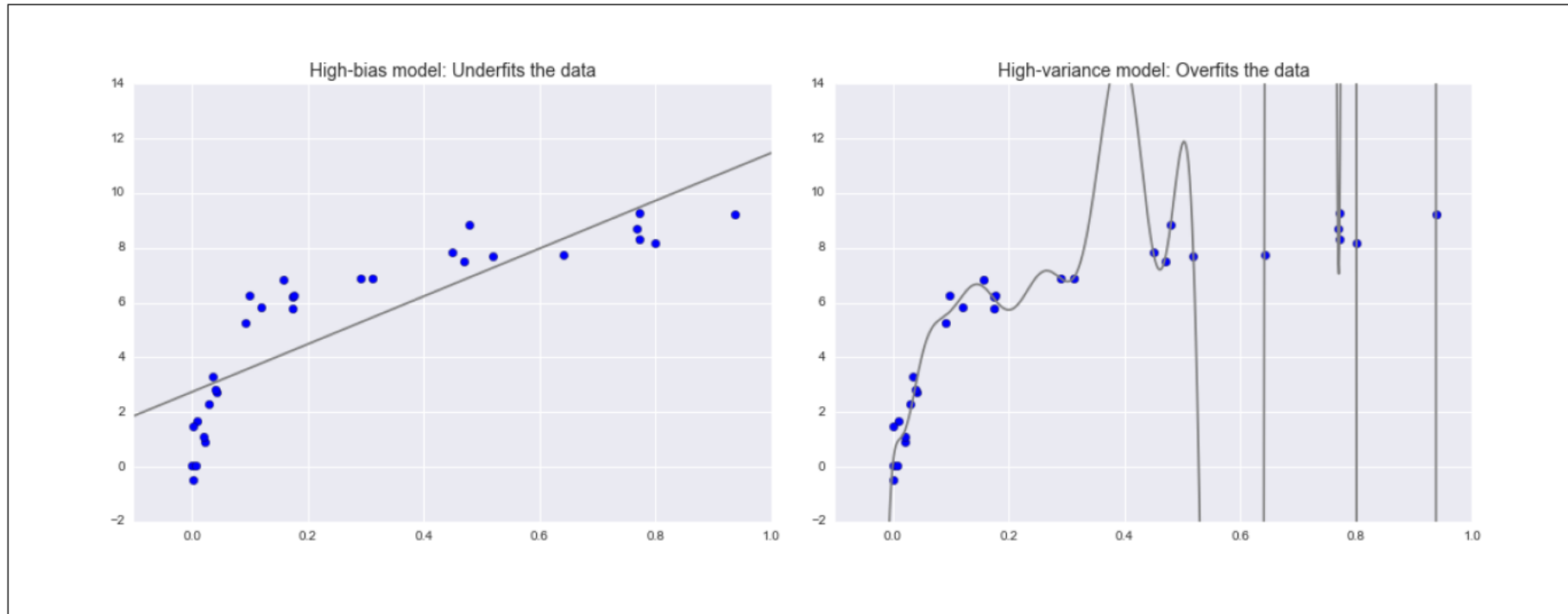
How to build and choose a model?

Split the data into:

- **Training set** (to learn the objective function)
- **Validation set** (to select the best model)
- **Test set** (to measure performance on unseen data)

If possible, maintain **proportions** among classes

Bias-Variance Tradeoff - Underfitting Vs. Overfitting

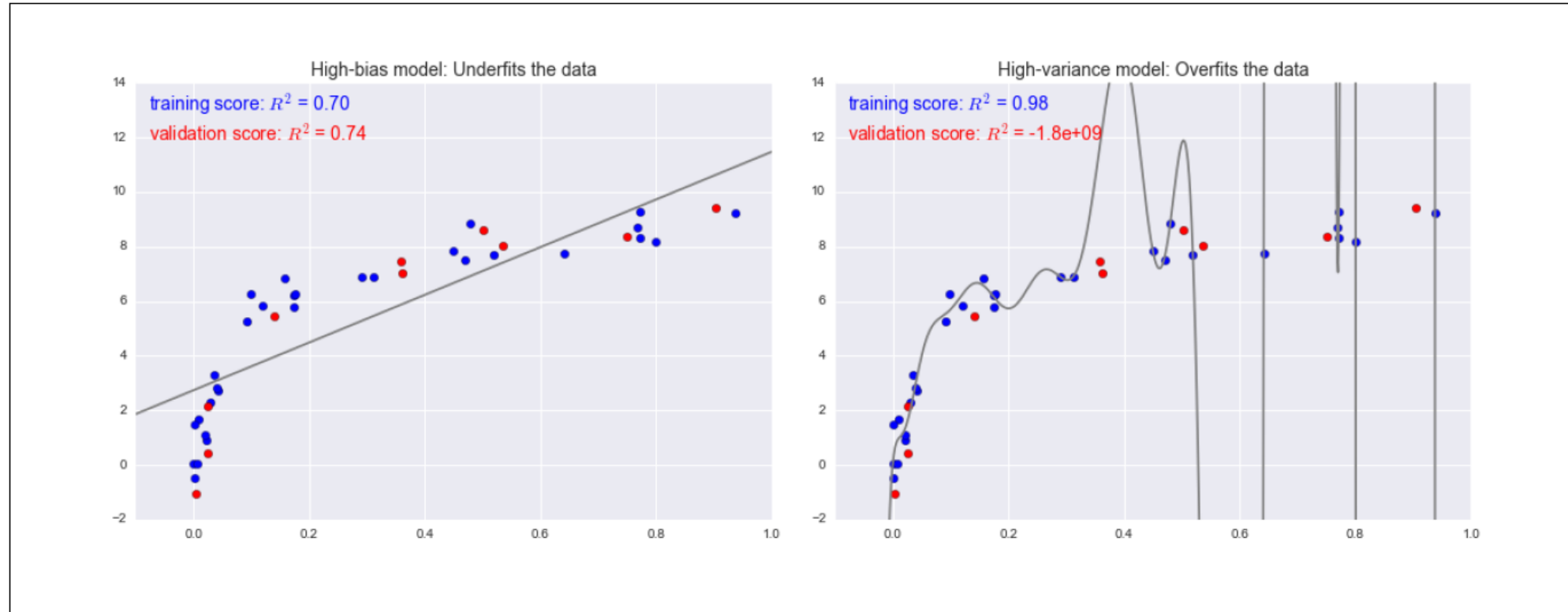


Model has **not enough flexibility**
It **underfits** the data
It has **high bias**

Model has **too much flexibility**
(that can account also for noise)
It **overfits** the data
It has **high variance**

What happens if new data come around?

What are the performance with new data?



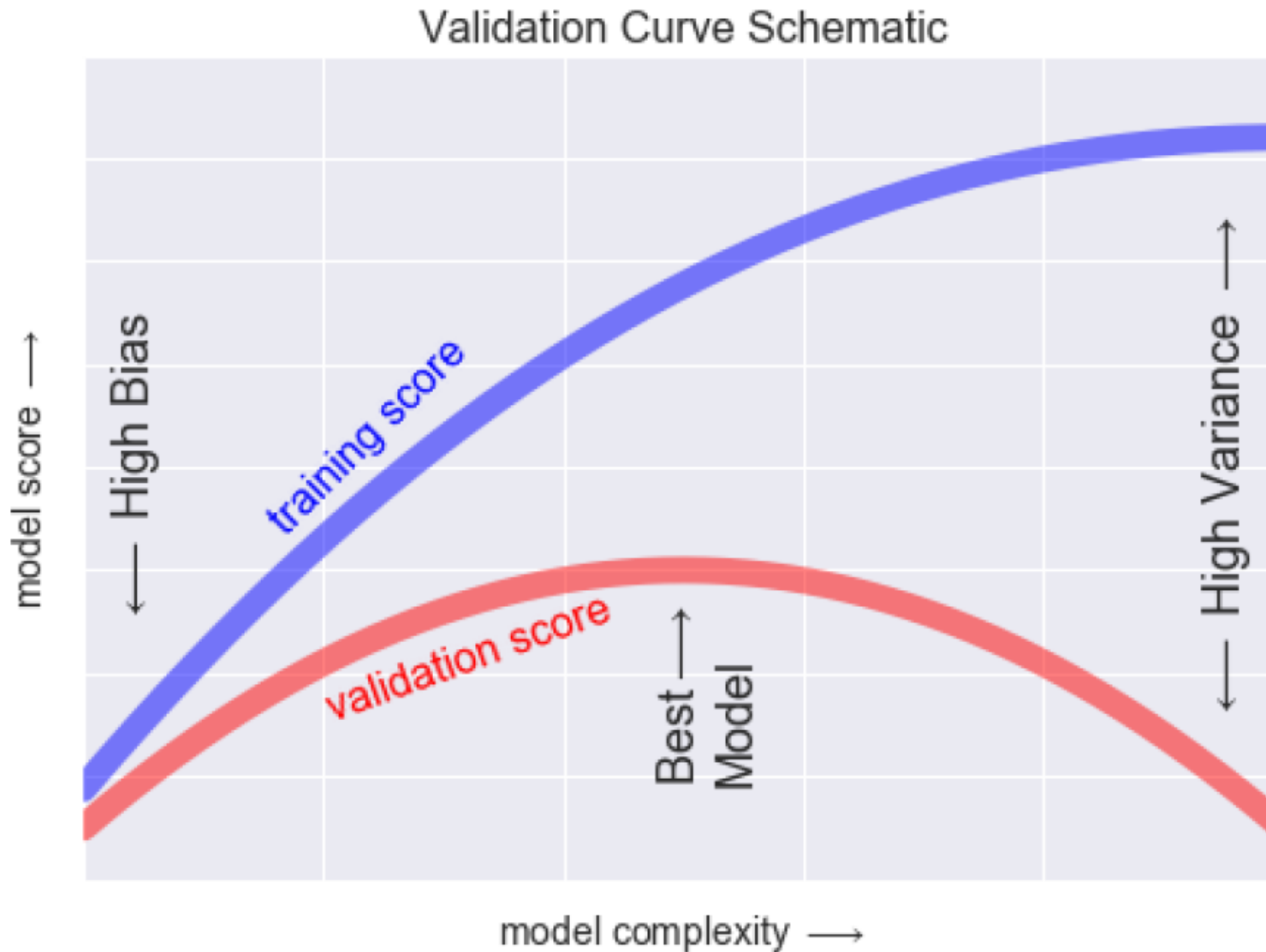
For high-bias models, the performance of the model on the validation set is similar to the performance on the training set

For high-variance models, the performance of the model on the validation set is far worse than the performance on the training set

Training, Validation (Testing) Set

- Therefore, it is important to evaluate the model on data different than the one used for training.
- Part of the data is used to train (fit) the model
- Part of the data is used for evaluation

Bias-Variance Error



K-Fold Cross Validation

Split the data into K sets, named **folds**

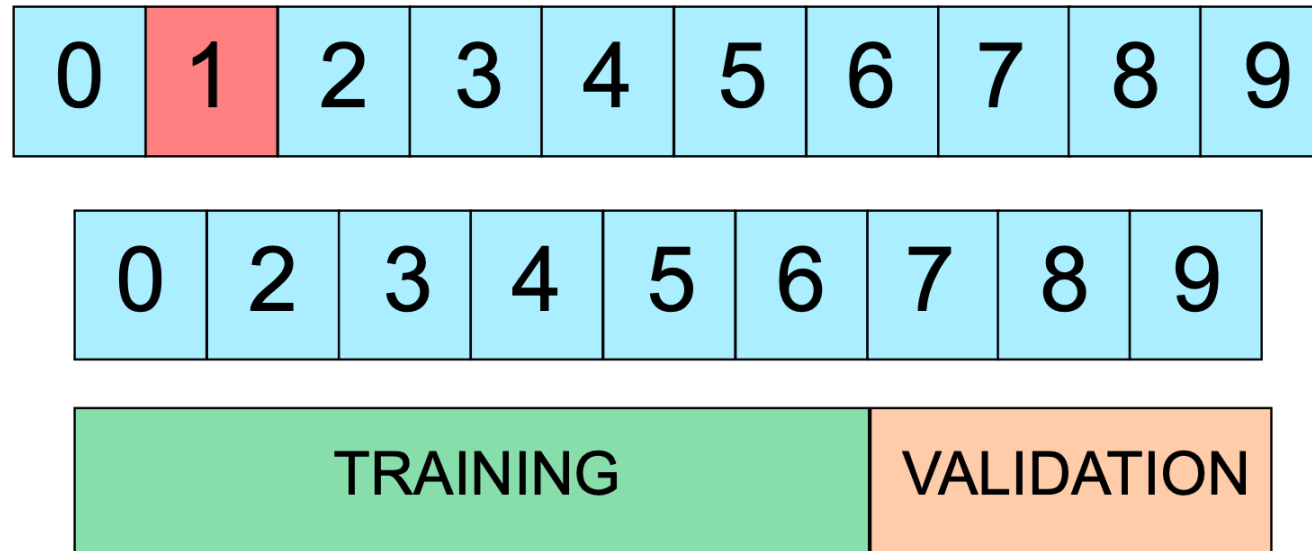
0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

In turn, each fold will act as **test set**, whereas the remaining will form the **training and validation sets**

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

K-Fold Cross Validation

Training and validation sets are used to select the model. Test set is used to compute the performance



K-Fold Cross Validation

The procedure is repeated K times, for all the folds

0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

Macro- vs. Micro-Average



Statistics computed **over the K folds** are reported.

Suppose to have the following results from a 3-fold cv:

- TP=5, TN=10, FP=2, FN=3
- TP=3, TN=12, FP=4, FN=1
- TP=8, TN=11, FP=0, FN=1

Two possibilities:

- compute one metric per fold, then average (**macro**)
- sum all TP, FP, FN, then compute one metric (**micro**)

Macro- vs. Micro-Average

Example — Macro-Precision

- TP=5, TN=10, FP=2, FN=3
- TP=3, TN=12, FP=4, FN=1
- TP=8, TN=11, FP=0, FN=1

$$P_1 = 5/7 = 0.714$$

$$P_2 = 3/7 = 0.286$$

$$P_3 = 8/8 = 1.000$$

$$P_{\text{Macro}} = (0.714 + 0.286 + 1.000) / 3 = \mathbf{0.667}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

Macro- vs. Micro-Average



Example — Micro-Precision

- TP=5, TN=10, FP=2, FN=3
- TP=3, TN=12, FP=4, FN=1
- TP=8, TN=11, FP=0, FN=1

$$P_{\text{micro}} = 16/22 = \mathbf{0.727}$$

Overfitting



A machine learning system is usually trained to **minimize some cost (loss) function**, thus it should not be surprising that it is capable to have a low error on the examples seen during training!

Main drawback

- One could **perfectly** learn the training set, while being completely **unable to generalize** across different examples!
- This is called **overfitting**

Overfitting



How to reduce overfitting?

- Validate model on data **not seen** during training
- Include a **regularization** term (more on this later)

Data Preprocessing



A crucial step in machine learning is to **prepare data** so that the **useful** information can be easily exploited

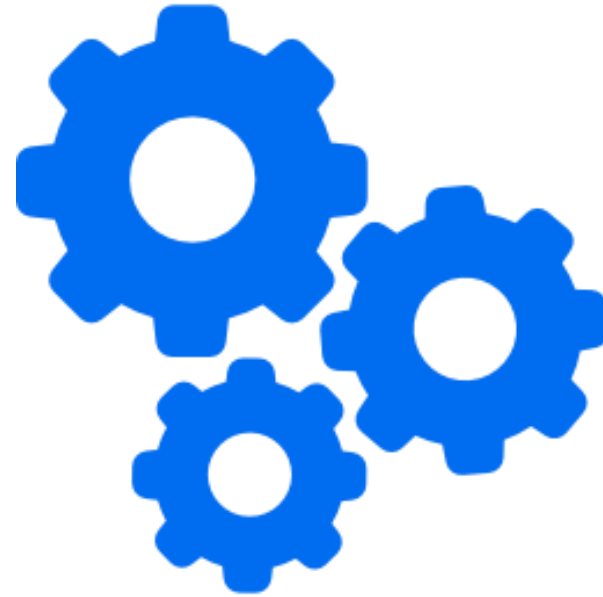
Data have to be formatted according to:

- Model/algorithm
- Software tool

Data Preprocessing

Main issues

- Integration
- Cleaning
- Normalization
- Aggregation
- Conversion
- Discretization
- Dimensionality reduction



Data Preprocessing

Integration

- Combine **several** different data sources
- Convert one **format** into another
- Select or **sample** data, if necessary
- Handle **units** of measurement
- Remove **duplicates**



[Image from naniinfotech.com]

Data Preprocessing

Cleaning (or cleansing)

- Handle **heterogeneous** values
- Handle **missing** data
 - **Ignore**: some models/algorithms can handle them
 - **Remove**: drop either examples or attribute
 - **Substitute**: with average/median/mode
- Check for **inconsistent** data (e.g. city/zipcode)
- Check for **outliers** (maybe due to mistakes)



Data Preprocessing



Normalization

- Some algorithms need the **input data** to be in a specific range of values (e.g. $[0,1]$ or $[-1,1]$)
- In general, it is a good practice to re-scale attribute values so that they are in **similar ranges**
- **Avoid** the fact that some attribute is **numerically too small/large** with respect to the others

Data Preprocessing



Min-Max normalization

$$\hat{v} = \frac{v - v^{\min}}{v^{\max} - v^{\min}}$$

Mean-Std normalization (a.k.a. standardization)

$$\hat{v} = \frac{v - \mu}{\sigma}$$

Data Preprocessing



Aggregation

- In many cases one needs to **group** some data according to some attribute value
- Typical in time-series, where one can choose the data **granularity** to be used for data representation

Data Preprocessing



Conversion

- Many algorithms can only handle **numeric** values
- Some attributes are **discrete** or **categorical**
- In the first case convert to **numeric** (ranked) values
- In the second case exploit **one-hot encoding**

Data Preprocessing

Ordinal to numeric conversion

- Preserve the **ordering** between discrete values

Example (Google traffic colormap):

Black
Red
Orange
Yellow
Green



0
1
2
3
4

(a)

1 0 0 0 0
1 1 0 0 0
1 1 1 0 0
1 1 1 1 0
1 1 1 1 1

(b)

0.0
0.25
0.5
0.75
1.0

(c)

Data Preprocessing

One-hot encoding

- Transform a **categorical** attribute with N possible values into a set of N **binary flags** (0/1)
- Example:

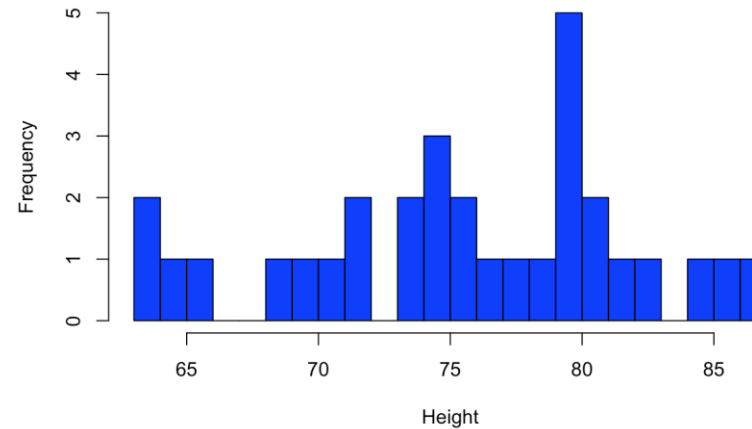
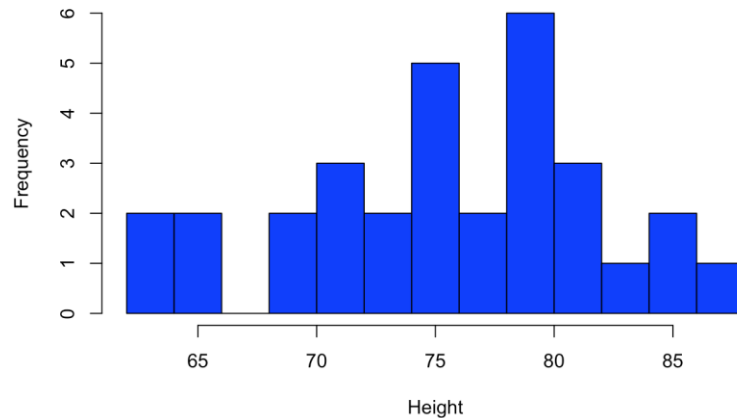
France	1 0 0 0 0
Italy	0 1 0 0 0
Germany	0 0 1 0 0
USA	0 0 0 1 0
UK	0 0 0 0 1

Note: all values are equally distant one from the other

Data Preprocessing

Discretization

- Some algorithms **cannot** handle numerical features
- Necessary to perform **quantization**
- Choose number of **bins** in the histogram



Data Preprocessing

Discretization

- **Equal-width binning:** partition the range into K bins, each having the same width
- **Equal-frequency binning:** partition the range into K bins, each having the same number of instances

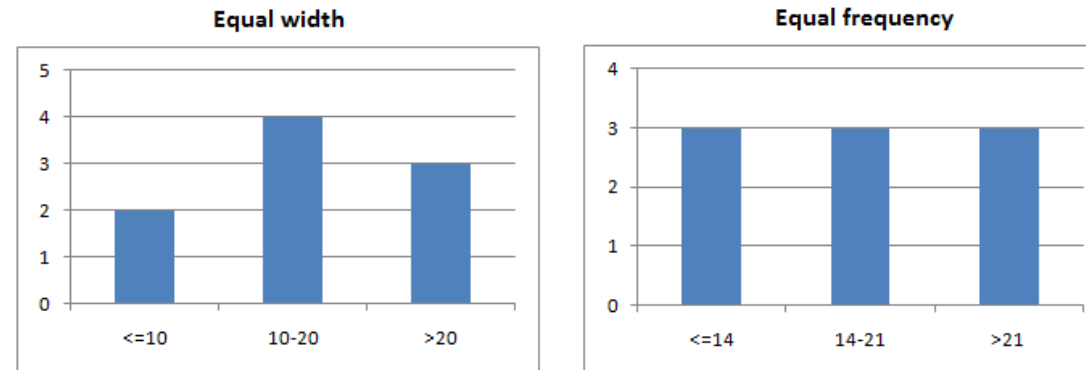


Image from <http://www.saedsayad.com/>

Data Preprocessing

- Dimensionality Reduction
 - PCA
 - Autoencoder
 - ...
- **Principal Component Analysis (PCA)** is a technique for mapping a set of examples, represented by many features, to a lower-dimensional space, where the main data characteristics are preserved. It is particularly useful for exploratory data analysis, for example to visualize data with many features

